ChatGPT

Post-Lecture Quiz: Advanced Optimizer Concepts (EECS 182)

Questions

1. **Local Linear Perspective:** What does it mean to view gradient descent from a local linear (first-order) perspective of the loss function? Why is gradient descent called a "first-order" optimization method?

2. **SignSGD and Norms:** What is sign SGD (sign-based SGD), and how is its update direction related to the **ℓ∞ norm**? Briefly explain why using the sign of the gradient corresponds to a particular notion of "steepest descent."

3. **RMS Norm & Xavier:** Define the root-mean-square (RMS) norm of a vector of weights. How does Xavier-style weight initialization use this concept to help maintain stable activations in a neural network?

4. **Spectral Norm Definition:** What is the spectral norm of a weight matrix, and why is it considered an **induced** matrix norm? In practical terms, what does the spectral norm tell us about how a layer affects its input?

5. **Geometry of SignSGD:** From a geometric standpoint, how does a **signSGD** update differ from a standard gradient descent update in weight space? (Hint: think about the shape of the "unit ball" in an ℓ∞ norm vs. an ℓ2 norm, and where the update vector lies in those shapes.)

6. **Xavier Intuition:** Xavier initialization is designed to "keep each layer at a reasonable scale." Explain in intuitive terms how choosing the right weight scale (via fan-in/fan-out) prevents activations from exploding or vanishing. Why is the RMS norm a natural measure to use for this purpose?

7. **Shampoo and $U!V^T$ Update:** In the **Shampoo optimizer**, we consider the SVD of a gradient matrix (or a second-moment matrix) for weights: $∇_W \ell = U \, Σ \, V^T$. One approximation used is to drop the Σ and update as $\Delta W \approx -\eta \, U V^T$. **Why does ignoring the singular values ($Σ$) make sense in this update?** What geometric problem is this solving compared to plain gradient descent?

8. **Spectral vs. Average Norms:** Controlling a weight matrix's spectral norm is stricter than just controlling its RMS (or Frobenius) norm. **What could go wrong if a matrix has a normal-looking RMS norm but a very large spectral norm?** Why might we care about spectral norm in addition to average norm when training deep networks?

9. **Connecting Xavier, RMS, and Spectral:** Fan-in initializations like Xavier keep the "average" effect of weights under control. How does this relate to the RMS norm of weight vectors, and what limitation does it have that a spectral norm constraint could address? (In other words, compare

what Xavier/RMS normalization handles vs. what spectral normalization handles in terms of signal magnitudes.)

10. **Shampoo's Second-Order Insight:** Standard gradient descent treats all directions in weight space the same. **How does Shampoo's preconditioning strategy use second-order information to adjust for the curvature or stretching of the loss landscape?** Describe what it means for Shampoo to make the update "more balanced across directions" in the context of an ill-conditioned (stretched) surface.

11. **µP for Scaling:** Maximal Update Parametrization (µP) is introduced to help with scaling up neural networks. **What problem does µP address when increasing model width, and how does it resolve it?** (Hint: consider hyperparameter tuning and the behavior of gradients/updates as networks grow larger.)

12. **Muon and Large-Scale Training:** The **Muon optimizer** builds on ideas from Shampoo and µP for large language models (LLMs). What are two key ways Muon improves training for very large models? Explain how Muon leverages **norm-based geometry** (like spectral normalization) and **µP-style scaling** to enable faster or more stable LLM training without extensive retuning.

Answer Key

1. **Local Linear Perspective:** Viewing gradient descent as a local linear method means at each step we approximate the loss surface by its tangent plane (using the first-order gradient). In other words, we assume the function is roughly linear in a small neighborhood around the current point and move in the direction of the negative gradient (the slope). Gradient descent is "first-order" because it uses only the first derivative (gradient) information and ignores curvature (second-order effects) when deciding the update.

2. **SignSGD and ℓ∞ Norm:** Sign SGD is a variant of SGD that uses only the sign of each gradient component (positive or negative) to update, instead of the full gradient magnitude. Geometrically, this means the update direction is **aligned with the gradient's sign pattern** but with equal influence from each coordinate. This corresponds to steepest descent under an ℓ∞ norm constraint – effectively, signSGD picks a direction to move that maximizes descent if your step can only have a limited maximum per-coordinate change [1]. In simpler terms, using the sign ignores the size of each gradient component and just uses the direction (±), which is optimal if you measure step size by the largest absolute component (the ℓ∞ norm).

3. **RMS Norm & Xavier:** The RMS norm of a vector is the root-mean-square of its components: for weights $w = (w_1,\dots,w_n)$, $ ‖ w ‖ {\text{RMS}} = \sqrt{\frac{1}{n}\sum$. It essentially measures the }^n w_i^2typical size of an entry in the vector. Xavier initialization uses fan-in (and sometimes fan-out) to choose weight variances such that the RMS of the weights is about $1/\sqrt{\text{fan-in}}$ (or a similar scale) [2]. This keeps the average magnitude of weights small enough that when many are summed in a neuron, the output stays in a reasonable range. In effect, Xavier init ensures that on **average**, each layer neither amplifies nor diminishes the scale of activations too much. By keeping the RMS norm of weights appropriately scaled, the network's signals neither explode nor vanish as they propagate through layers.

4. **Spectral Norm Definition:** The spectral norm of a matrix (often denoted $ ‖ W ‖ 2$) is the largest singular value of that matrix. It's called an **induced** norm because it equals the maximum stretch factor the matrix applies to any input vector in the corresponding vector norm (for spectral norm,

the vector norm is Euclidean). In practical terms, the spectral norm tells us the worst-case amplification of the input: if $ ‖ W ‖ _2 = \sigma$, then for any unit input x we have $ ‖ W x ‖ 2 \le \sigma$. It measures the maximum "stretching" of the "rubber sheet" mapping caused by W [3]. A large spectral norm means the layer can greatly magnify some particular direction of input, which is important for understanding stability (e.g. risk of exploding activations or gradients along some direction).

5. **Geometry of SignSGD:** Geometrically, signSGD chooses an update that points to the **corner of a hypercube** in weight space, whereas standard gradient descent points along the exact gradient vector. In an ℓ∞ norm "unit ball" (a hypercube), the extreme directions are those that push each coordinate to its limit (±1 in each axis) – signSGD's update goes toward one of these corners (each weight step is ±η) [4]. In contrast, standard gradient descent is steepest in an ℓ2 sense, so its step lies on the surface of a sphere (ℓ2 ball) in the precise direction of the gradient vector. The result is that **signSGD treats all coordinates uniformly** (only direction matters, not magnitude), moving along a path aligned with axes, whereas standard GD's step is proportional to the gradient magnitude in each direction (a "straight" direction through the sphere).

6. **Xavier Intuition:** Xavier initialization keeps each layer's weights at a scale where the outputs neither blow up nor die out. Intuitively, if you imagine pouring "signal energy" through each layer, Xavier makes sure each layer passes that energy along without adding or removing too much. By setting weight variance $\sim 1/(\text{fan-in})$, the sum of many inputs (each scaled by a weight) tends to have a manageable variance. In terms of RMS norm, Xavier is ensuring the typical weight magnitude is about $1/\sqrt{d_{\text{in}}}$ [2], so that when you multiply by d_{in} inputs (of order 1 each), the output stays in the same ballpark. This prevents activation distributions from growing wider (exploding) or collapsing to zero (vanishing) as they propagate. The RMS norm is a natural measure here because it accounts for the number of inputs: by dividing by $\sqrt{d_{\text{in}}}$, it fairly measures the size of each weight in a layer-size-neutral way (so having 1000 small weights is comparable to 10 larger weights in terms of effect).

7. **Shampoo and $U V^T$ Update:** Dropping $Σ$ in the Shampoo update (using $\Delta W \approx -\eta\, U V^T$ when $∇W = U Σ V^T$) means we **ignore the singular values**, i.e. the stretching magnitudes, and keep only the directional information from the gradient [1] [5]. This makes sense because $Σ$ often reflects how distorted or ill-conditioned the geometry is at W, rather than the true importance of each direction for reducing loss. In plain gradient descent, if the weight space is highly stretched (like a trampoline pulled unevenly), the raw gradient in some directions can be huge (if the surface is steep and stretched there) and tiny in others (if compressed), even if those directions are equally important for optimization [6]. By using UV^T (dropping $Σ$), Shampoo **normalizes away these stretch effects** – every principal direction of $∇W$ gets treated more equally. Geometrically, it's as if we're saying "don't over-trust the raw slope's magnitude when the ground is warped." The update $-η\,UV^T$ still points in the combined "steepest descent" direction of the gradient **but rescales the step so that no direction gets an excessively large or tiny update just because of conditioning** [7]. This helps fix the imbalance that plain gradient descent would suffer on an ill-conditioned surface, making the step fairer across all directions.

8. **Spectral vs. Average Norms:** If a weight matrix has a normal RMS norm but a very large spectral norm, it means that on average the weights aren't huge, **but there might be a particular direction where the matrix's effect is excessively strong**. In such a case, most inputs might behave fine, yet an input aligned with that special direction could get its signal enormously amplified. This is dangerous because it can lead to unstable activations or gradients (for example, one neuron combination could blow up even though overall variance looks fine). Thus, just

controlling the average norm (RMS/Frobenius) isn't enough to guarantee stability in the worst case. We care about spectral norm because it caps that worst-case amplification – by ensuring $\| W \|_2$ isn't too large, we guarantee no direction will be stretched beyond what we expect. In practice, methods like spectral normalization or Shampoo's updates explicitly address this: they make sure that **even outlier directions are kept in check**, complementing the average control from approaches like Xavier.

9. **Connecting Xavier, RMS, and Spectral:** Xavier initialization (and similar schemes) addresses the **average-case** scaling: it uses RMS norm considerations to ensure that each layer, on average, preserves the scale of signals. This solves the problem of signal variance blowing up or decaying by tailoring the initial weight RMS to the layer size. However, Xavier by itself doesn't guarantee anything about a single worst-case direction. A spectral norm constraint picks up where Xavier leaves off: it ensures no singular direction of the weight matrix can amplify signals too much. In other words, Xavier keeps the typical activation scale reasonable (on average across neurons), while spectral normalization would ensure **even the most "dangerous" direction is under control**. The limitation of only using an average (RMS) norm is that a matrix could still have one extremely large singular value (causing instability) balanced out by many small ones; a spectral constraint would catch and limit that. Together, using RMS-based initialization and then perhaps bounding spectral norm can yield both overall stable propagation and guaranteed per-direction stability [3].

10. **Shampoo's Second-Order Insight:** Shampoo uses the idea of preconditioning the gradient with an approximate inverse curvature matrix. This means it doesn't take the gradient at face value in each direction; instead, it scales different directions differently based on how the loss landscape is shaped (the curvature or Hessian-like information). If the loss surface is an elongated bowl (ill-conditioned), some directions are "steeper" just because the landscape is stretched in those directions. Shampoo adjusts for that by effectively **whitening or isotropizing the gradient** – directions that were stretched (where a small parameter change causes a big loss change) get a smaller step, and directions that were compressed get a bigger step. In practical terms, Shampoo makes the update more balanced across directions by using second-order information (like the gradient's covariance or its SVD) to normalize the step sizes. It's as if it unstretches the landscape in the gradient's eye: a step on an unevenly stretched rubber sheet becomes more like a step on an evenly stretched sheet [6] [7]. This leads to faster convergence on ill-conditioned problems, since no single direction's curvature can bottleneck the learning rate – each principal direction of the weight update is scaled appropriately.

11. **µP for Scaling:** Maximal Update Parametrization (µP) addresses the problem that as we increase a network's width, the optimal hyperparameters (like learning rate, weight initialization scale, etc.) often change. With standard parametrization, a model that is 10× wider might require a smaller learning rate or adjusted initialization to train well. µP provides a recipe for scaling the initialization and the learning rate with width such that **the training dynamics remain stable and consistent** as the model grows [8]. In essence, µP ensures that gradients and updates do not vanish or explode with width – so a wide model and a narrow model behave similarly when using the same optimizer settings. This means you can use the **same hyperparameters for small models and large models** without retuning, because the updates per parameter (the "maximal update") are kept on a similar scale. It greatly simplifies scaling: you can find a good learning rate on a smaller model and trust that µP will make it work for a much larger model, avoiding the need for exhaustive hyperparameter search when increasing model size.

12. **Muon and Large-Scale Training:** Muon is a recently introduced optimizer that builds on Shampoo's geometry-aware updates and µP's scaling principles to excel at large-scale training

[9]. First, Muon incorporates **spectral normalization-like preconditioning** (similar to Shampoo) – it orthogonalizes or normalizes gradient updates so that all directions in the weight matrix are treated appropriately. This helps large models train faster and more stably by addressing ill-conditioned curvature (no single direction blows up the update). Second, Muon integrates **μP-style consistent scaling of updates** [10], which means the optimizer's behavior remains stable as the model size grows (you don't need to retune learning rates for different model widths). In practice, Muon's design leads to significantly faster training (it achieved >10× speedups in some LLM training scenarios by enabling larger, balanced steps) and it maintains training stability without extensive hyperparameter tuning when moving to bigger models [11] [12]. In short, Muon "cleans up" the geometry of the optimization like Shampoo and ensures hyperparameters transfer across scales like μP, making it highly effective for huge networks.

---

[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] Special Participation E Intuitively review optimizer with Claude.pdf

file://file_00000000e0d8722f8d653e2e4a8dfba5